Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр

Тема: «Среда программирования C++. Разработка и документирование консольных приложений. Классы»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Познакомиться с интегрированной средой разработки; изучить технологию структурной разработки программных средств и перенос логики задания в среду программирования C++; освоить базовые элементы объектно-ориентированного программирования: классы, конструкторы, методы, массивы как члены класса, перегрузку операторов.

Ход работы:

Задание: Выбрать предметную область и реализовать на её основе пользовательский класс с тремя свойствами-атрибутами (одно из них - статический массив целых чисел фиксированного размера), тремя видами конструкторов (по умолчанию, с параметрами, копирования), методами `TO_SHOW`, `TO_GET_<ИмяСвойства>`, `TO_SET_<ИмяСвойства>` и тремя перегруженными операторами (присваивание =, бинарный и унарный). Продемонстрировать работу класса в функции `main()`.

Описание предметной области и разработанного класса:

Выбранная предметная область - **Студент (Student)**. Класс `Student` описывает студента университета: курс, номер группы и оценки за последнюю сессию. Все данные класса закрыты (`private`), методы открыты (`public`). Динамические массивы, указатели и контейнеры STL не используются.

Поля класса:

- `int course` - номер курса (1, 2, 3...);
- `int group` - номер группы;
- `int grades[SIZE]` - статический массив из пяти оценок за сессию.

Конструкторы:

- `Student()` - по умолчанию: курс = 1, группа = 0, все элементы массива равны нулю;
- `Student(int c, int g, const int arr[])` - с параметрами: принимает курс, группу и массив из `SIZE` оценок, копирует элементы поэлементно;
- `Student(const Student& other)` - копирования: создаёт независимую копию объекта, включая все элементы массива.

Методы:

- `TO_SHOW()` - выводит всю информацию об объекте;
- `TO_GET_Course()`, `TO_GET_Group()` - возвращают значения курса и группы;
- `TO_GET_Grade(int i)` - возвращает i-ю оценку (при неверном индексе возвращает -1);
- `TO_SET_Course(int c)`, `TO_SET_Group(int g)` - устанавливают курс и группу;
- `TO_SET_Grade(int i, int value)` - устанавливает i-ю оценку (при неверном индексе ничего не меняет).

Перегруженные операторы:

- `operator=` - присваивание: копирует все поля, включая поэлементное копирование массива оценок; защищён от самоприсваивания, возвращает ссылку на текущий объект (допускает цепочку `a = b = c`);
- `operator+` (бинарный) - «объединение» двух студентов: оценки складываются поэлементно (суммарные баллы по каждому из пяти предметов), курс берётся наибольший из двух, группа - от левого операнда. Результат возвращается как новый объект по значению, исходные операнды не изменяются;
- `operator++` (унарный, префиксный) - перевод студента на следующий курс: увеличивает `course` на 1 и возвращает ссылку на изменённый объект.

Код программы:

```
#include <iostream>
#include <cstdlib>
#include <clocale>
using namespace std;

class Student
{
public:
    static const int SIZE = 5;
```

```
private:
    int course;
    int group;
    int grades[SIZE];

public:
    Student()
    {
        course = 1;
        group = 0;
        for (int i = 0; i < SIZE; i++)
        {
            grades[i] = 0;
        }
    }

    Student(int c, int g, const int arr[])
    {
        course = c;
        group = g;
        for (int i = 0; i < SIZE; i++)
        {
            grades[i] = arr[i];
        }
    }

    Student(const Student &other)
    {
        course = other.course;
        group = other.group;
        for (int i = 0; i < SIZE; i++)
        {
            grades[i] = other.grades[i];
        }
    }

    void TO_SHOW() const
    {
        cout << "Курс: " << course;
        cout << " | Группа: " << group;
        cout << " | Оценки: ";
        for (int i = 0; i < SIZE; i++)
        {
            cout << grades[i] << " ";
        }
        cout << endl;
    }

    int TO_GET_Course() const { return course; }

    int TO_GET_Group() const { return group; }

    int TO_GET_Grade(int i) const
```

```
{
    if (i < 0 || i >= SIZE)
        return -1;
    return grades[i];
}

void TO_SET_Course(int c) { course = c; }

void TO_SET_Group(int g) { group = g; }

void TO_SET_Grade(int i, int value)
{
    if (i >= 0 && i < SIZE)
    {
        grades[i] = value;
    }
}

Student &operator=(const Student &other)
{
    if (this != &other)
    {
        course = other.course;
        group = other.group;
        for (int i = 0; i < SIZE; i++)
        {
            grades[i] = other.grades[i];
        }
    }
    return *this;
}

Student operator+(const Student &other) const
{
    Student result;
    if (course > other.course)
    {
        result.course = course;
    }
    else
    {
        result.course = other.course;
    }
    result.group = group;
    for (int i = 0; i < SIZE; i++)
    {
        result.grades[i] = grades[i] + other.grades[i];
    }
    return result;
}

Student &operator++()
{
```

```
        course++;
        return *this;
    }
};

int main()
{

    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    cout << "=== Создание объектов ===" << endl;

    Student s1;
    cout << "Объект 1 (по умолчанию): ";
    s1.TO_SHOW();

    int g[Student::SIZE] = {8, 9, 7, 10, 9};
    Student s2(2, 29, g);
    cout << "Объект 2 (с параметрами): ";
    s2.TO_SHOW();

    Student s3(s2);
    cout << "Объект 3 (копия объекта 2): ";
    s3.TO_SHOW();

    cout << endl
         << "=== Методы TO_SET ===" << endl;

    s1.TO_SET_Course(1);
    s1.TO_SET_Group(15);
    int newGrades[Student::SIZE] = {6, 7, 5, 8, 6};
    for (int i = 0; i < Student::SIZE; i++)
    {
        s1.TO_SET_Grade(i, newGrades[i]);
    }
    cout << "Объект 1 после TO_SET: ";
    s1.TO_SHOW();

    s3.TO_SET_Grade(4, 5);
    cout << "Объект 3 после TO_SET_Grade(4, 5): ";
    s3.TO_SHOW();
    cout << "Объект 2 (не изменился): ";
    s2.TO_SHOW();

    cout << endl
         << "=== Методы TO_GET ===" << endl;
    cout << "Курс объекта 2: " << s2.TO_GET_Course() << endl;
    cout << "Группа объекта 2: " << s2.TO_GET_Group() << endl;

    cout << "Оценка №4 объекта 2: ";
    cout << s2.TO_GET_Grade(3) << endl;
```

```
    cout << endl
         << "=== Оператор = ===" << endl;
    s1 = s2;
    cout << "Объект 1 после s1 = s2: ";
    s1.TO_SHOW();

    cout << endl
         << "=== Бинарный оператор + ===" << endl;

    Student s4 = s2 + s3;
    cout << "Результат s2 + s3: ";
    s4.TO_SHOW();

    cout << endl
         << "=== Унарный оператор ++ ===" << endl;
    cout << "Объект 3 до ++: ";
    s3.TO_SHOW();
    ++s3;
    cout << "Объект 3 после ++: ";
    s3.TO_SHOW();

    system("pause");
    return 0;
}
```

Результат:

```
Active code page: 65001
=== Создание объектов ===
Объект 1 (по умолчанию): Курс: 1 | Группа: 0 | Оценки: 0 0 0 0 0
Объект 2 (с параметрами): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9
Объект 3 (копия объекта 2): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Методы TO_SET ===
Объект 1 после TO_SET: Курс: 1 | Группа: 15 | Оценки: 6 7 5 8 6
Объект 3 после TO_SET_Grade(4, 5): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5
Объект 2 (не изменился): Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Методы TO_GET ===
Курс объекта 2: 2
Группа объекта 2: 29
Оценка №4 объекта 2: 10

=== Оператор = ===
Объект 1 после s1 = s2: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 9

=== Бинарный оператор + ===
Результат s2 + s3: Курс: 2 | Группа: 29 | Оценки: 16 18 14 20 14

=== Унарный оператор ++ ===
Объект 3 до ++: Курс: 2 | Группа: 29 | Оценки: 8 9 7 10 5
Объект 3 после ++: Курс: 3 | Группа: 29 | Оценки: 8 9 7 10 5
Press any key to continue . . .
```

Вывод: Познакомились со средой программирования C++, разработали и документировали консольное приложение. Освоили базовые элементы объектно-ориентированного программирования: создали класс `Student` с закрытыми полями (в том числе статическим массивом), реализовали конструкторы по умолчанию, с параметрами и копирования, методы доступа `TO_SET`, `TO_GET`, `TO_SHOW`, а также перегрузили оператор присваивания, бинарный оператор + и унарный оператор ++.